Elizabeth's three-day interview study edition

AI/ML Interview Flashcards

Equations, notation, and intuitive explanations

157 numbered cards • Six card colors

CNNs / SVMs / Algorithms / Graphs / Software / Applied ML

Your card-stock map

White takes the largest stack. The five colored groups are **pink, yellow, orange, green, and blue**. This replaces the earlier purple suggestion. Pale printed tints identify groups without sacrificing contrast; the physical stock is your stronger visual cue.

Stock	Cards	Topic
White	75	Foundations, learning, OOD/OOP, APIs, testing
Yellow	20	Algorithms, sorting, recursion, DP
Blue	15	Graphs, Dijkstra, A*, graph complexity
Green	17	Queues, heaps, arrays, lists, maps, trees
Pink	15	Deep learning, especially CNNs
Orange	15	NLP, generation, recommendation, RL

Priority marker: a black star identifies a highlighted screenshot topic or a direct supporting concept. It is your supplied emphasis, not a prediction of interview frequency.

Scope: covers the readable topics in your three screenshots. Crossed-out speech/audio processing is omitted. "Practical and theory" is treated as understanding both mechanisms and use cases.

How to use a card: front = term plus a question. Back = meaning, equation, variable definitions, and a caveat. Long cards are expanded study backs, not a requirement to hand-copy every sentence. Reconstruct equations aloud before checking them.

Two independent color systems

Card-stock color says which topic you are in. Pen color says what a mark means. The footer names all six topics and the current stock color on every page.

GREEN = GO: predictions, useful intuition, valid updates, and invariants. Underline what moves the solution forward. Use a dark green on green stock; add the label GO if contrast is weak.

ORANGE = LEARN: losses, objectives, fitted quantities, and training choices. Put a small orange box around the loss or update being optimized. On orange stock, use dark green plus the label LEARN when orange ink is hard to see.

PINK = STOP: assumptions, failure modes, and easy confusions. On pink stock, switch this role to a dark orange or dark green pen and write STOP. Keep the meaning even when the ink changes.

Definitions and symbols: use black or your darkest readable pen. Do not color every symbol. A small label or underline is enough. Card numbers and GO/LEARN/STOP labels preserve meaning if a color is hard to distinguish.

Connections: write a small related-card number in the margin, such as "heap → Dijkstra" or "cross-entropy → language modeling." Use one consistent dark-green mark for these links. The shared mathematical mechanism matters more than matching topic colors.

Preparation evening + three study days

Tonight	Make starred cards first. Then make cards you cannot explain from memory. For familiar cards, copy only the equation, symbol meanings, and one caveat.
Day 1	Yellow, blue, green: trace sorting, BFS, heaps, Dijkstra, and A*. Code small examples and justify their complexity.
Day 2	White ML, pink, orange: derive core losses, explain SVM/CNN notation, calculate metrics, and describe your recommender.
Day 3	White software/testing plus mixed recall. Complete a timed coding problem and verbal ML mock; revisit missed cards after a break.

A successful recall: define the task, identify the inputs and outputs, write the main equation, explain its symbols, and give an assumption or counterexample. Mark answers as knew / partly knew / missed and spend more time on the last two groups.

Learning: inputs → model → loss → update

1. Supply examples and choose a model

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}, \qquad x_i \in \mathbb{R}^d, \qquad \hat{y}_i = f_\theta(x_i).$$

x_i contains the d feature values for example i; y_i is its observed outcome. The model family determines which input–output relationships are representable. The learned parameters θ select one member of that family.

2. Measure individual and average error

$$\ell_i(\theta) = \ell(y_i, f_\theta(x_i)), \qquad \mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\theta).$$

Lowercase ℓ is one example's loss. Calligraphic capital $\mathcal{L}$ is average data loss here. Other authors may use $\mathcal{L}$ for the entire regularized objective; define the convention explicitly.

3. Add a preference for simpler solutions

$$\boxed{\mathcal{J}(\theta) = \mathcal{L}(\theta) + \lambda\Omega(\theta), \qquad \hat{\theta} = \arg\min_\theta \mathcal{J}(\theta).}$$

Ω is a penalty, and λ controls its strength. The arg min returns minimizing parameter values; min returns the minimum objective value. If there is no regularization, $\lambda = 0$ and $\mathcal{J} = \mathcal{L}$.

4. Update parameters using a gradient

$$\boxed{\theta_{t+1} = \theta_t - \eta_t \nabla_\theta \mathcal{J}(\theta_t).}$$

For $\theta = (w_1, \ldots, w_d, b)$, the gradient stacks the partial derivatives with respect to every weight and the bias. A positive partial derivative causes a negative update to that parameter.

$$w_j = 2, \quad \eta = 0.1, \quad \partial\mathcal{J}/\partial w_j = 3 \quad \Rightarrow \quad w_j^{new} = 1.7.$$

STOP / caveat. This is a local downhill direction; an excessively large step can still increase the loss. For nonconvex objectives, training need not find a global minimum.

The gradient is evaluated at the current parameters

$$\nabla_\theta \mathcal{J}(\theta_t) = \begin{bmatrix} \partial \mathcal{J}/\partial w_1 \\ \vdots \\ \partial \mathcal{J}/\partial w_d \\ \partial \mathcal{J}/\partial b \end{bmatrix}\Bigg|_{\theta=\theta_t}.$$

Batch gradient descent uses every training example for each step. **Stochastic gradient descent** traditionally uses one; **minibatch SGD** uses a small set B_t:

$$\theta_{t+1} = \theta_t - \eta_t \left[\frac{1}{|B_t|} \sum_{i \in B_t} \nabla_\theta \ell_i(\theta_t) + \lambda \nabla_\theta \Omega(\theta_t) \right].$$

B_t contains example indices, $|B_t|$ is batch size, and t counts updates. One **epoch** is one pass through the training dataset; it usually contains many parameter updates.

Optimizer example: Adam

For gradient g_t at step t, Adam maintains moving averages:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

All squares, square roots, and division are elementwise. m_t tracks gradients; v_t tracks squared gradients; β_1, β_2 are decay rates; ϵ stabilizes division. Initialize the averages at zero. AdamW uses decoupled weight decay rather than simply treating it as an L2 gradient term.

Training objective versus the real goal

$$R(\theta) = \mathbb{E}_{(X,Y)\sim P_{deploy}}[\ell(Y, f_\theta(X))].$$

We can compute training loss directly. We estimate deployment risk using representative held-out data. A low training objective is useful only when its learned behavior transfers.

Notation habits that sound precise: say "Let $K(x, z)$ denote the kernel," or "Use NCHW with C_{out} filters." Define the symbol rather than implying it has one universal meaning. K can be a class count, SVM kernel, or attention-key matrix; W can be image width or learned weights.

WHITE STOCK Foundations, learning, software, and testing

Cards 1–75. Read the local notation before memorizing the equations.

$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$	Dataset; i indexes examples and n is their count.
$x_i \in \mathbb{R}^d,\ x_{ij},\ X \in \mathbb{R}^{n \times d}$	Feature vector; feature j of example i; matrix with examples as rows.
$y_i,\ \hat{y}_i,\ \hat{p}_i$	True target, predicted target, and predicted probability.
$f_\theta,\ \theta,\ \hat{\theta}$	Parameterized model, all learned parameters, and fitted parameters.
$w \in \mathbb{R}^d,\ b \in \mathbb{R},\ z = w^\top x + b$	Weights, bias/intercept, and raw score; z is a logit in logistic regression.
$\ell_i,\ \mathcal{L},\ \mathcal{J},\ \Omega$	Per-example loss, mean data loss, full objective, regularization penalty.
$\eta_t,\ \lambda,\ t,\ B_t$	Learning rate, regularization strength, update index, minibatch index set.
$\nabla_\theta \mathcal{J},\ \partial \mathcal{J} / \partial w_j$	Gradient vector and one partial derivative.
$\arg\min_\theta \mathcal{J},\ \min_\theta \mathcal{J}$	Minimizing parameters versus the minimum objective value.
$P,\ \mathbb{E},\ \mathrm{Var},\ \mathbf{1}[A]$	Probability distribution, expectation, variance, and indicator of event A.
$TP, FP, TN, FN,\ \tau$	Confusion-matrix counts and classification threshold.
$\top,\ \|w\|_2,\ \odot$	Transpose, Euclidean norm, elementwise product.

STOP / caveat. Notation is conventional, not universal. State your definitions; do not present one author's letters as a rule.

001 Supervised learning

Meaning. Learn a mapping from input features to observed targets using labeled examples. Classification predicts categories; regression predicts numerical outcomes.

$$\hat{\theta} = \arg\min_{\theta} \mathcal{J}(\theta), \qquad \mathcal{J} = \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f_\theta(x_i)) + \lambda\Omega(\theta).$$

Notation / context. θ: learned parameters; ℓ: example loss; $\lambda\Omega$: complexity penalty.
GO / intuition. Choose parameters that fit observed outcomes and generalize to future examples. See the learning walkthrough before Card 1.

002 Unsupervised learning

Meaning. Learn structure without supplying a target label for each example. The algorithm still has an explicit objective. For k-means clustering:

$$\min_{\mu_1,\ldots,\mu_K} \sum_{i=1}^{n} \min_{k\in\{1,\ldots,K\}} \|x_i - \mu_k\|_2^2.$$

Notation / context. K: cluster count; μ_k: centroid of cluster k. Alternate nearest-centroid assignment and centroid recomputation.
STOP / caveat. Unsupervised does not mean objective-free; clusters need not correspond to meaningful real-world categories.

003 Classification

Meaning. Predict a discrete label, usually by first computing class scores or probabilities and then choosing a class.

$$\hat{y} = \arg\max_{k\in\{1,\ldots,K\}} p_\theta(y = k \mid x).$$

Notation / context. K: number of classes; p_θ: model's conditional distribution. For binary classification, $\hat{y} = \mathbf{1}[\hat{p} \geq \tau]$.
GO / intuition. Separate the score, probability estimate, and final decision. A threshold changes the decision without retraining the scoring model.

004 Regression

Meaning. Predict a numerical outcome. Squared error is a common loss; absolute error is another. Their optimal population predictions differ.

$$\ell_2 = (y - \hat{y})^2, \qquad \ell_1 = |y - \hat{y}|.$$

Notation / context. Under squared loss the optimal prediction is $\mathbb{E}[Y \mid X = x]$; under absolute loss it is a conditional median.
STOP / caveat. A numerical class code is not automatically a regression target. Logistic regression is typically a classifier.

005 Features and labels

Meaning. Features describe what is available at prediction time; labels represent the outcome to predict. Write one example as a vector plus its target.

$$x_i = (x_{i1}, \ldots, x_{id})^\top, \qquad (x_i, y_i) \in \mathcal{D}.$$

Notation / context. i: example; j: feature; d: number of features. For fraud, amount and account age may be features; verified fraud is the label.
STOP / caveat. A value recorded only after the outcome is generally not a valid prediction-time feature.

006 Parameters versus hyperparameters

Meaning. Parameters are fitted from training data; hyperparameters control the model family or learning procedure and are usually selected using validation data.

$$\hat{\theta}(h) = \arg\min_{\theta} \mathcal{J}_{\text{train}}(\theta; h), \quad \hat{h} = \arg\min_{h} \mathcal{L}_{\text{val}}(\hat{\theta}(h)).$$

Notation / context. h: hyperparameter configuration, such as tree depth or learning rate; θ: weights, biases, or other fitted quantities.
STOP / caveat. Do not select hyperparameters by repeatedly checking the test set.

007 Train, validation, and test sets

Meaning. Training fits parameters; validation guides model choices; testing estimates performance after those choices.

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}.$$

Notation / context. These sets should not share examples; often they must also separate users, entities, duplicates, or time periods.
GO / intuition. Match the split to deployment. Predicting future outcomes usually requires chronological evaluation rather than a random split.

008 Cross-validation

Meaning. Rotate held-out folds to estimate a procedure's performance and variability. Each fit must include its own preprocessing and feature selection.

$$\widehat{R}_{\text{CV}} = \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}_{\mathcal{D}_k}(\hat{\theta}_{-k}).$$

Notation / context. $\mathcal{D}_k$: validation fold; $\hat{\theta}_{-k}$: model fitted on the other folds; K: fold count.
STOP / caveat. Random folds can leak future information or related entities. Use grouped or time-aware folds when required.

009 Generalization

Meaning. Performance on unseen examples from the relevant setting. Training minimizes an empirical quantity; deployment depends on a population quantity.

$$R(\theta) = \mathbb{E}_{(X,Y)\sim P_{\text{deploy}}}[\ell(Y, f_\theta(X))].$$

Notation / context. R: population risk; P_{deploy}: deployment distribution. An empirical generalization gap compares population and training risks.
GO / intuition. Representative held-out evaluation estimates generalization; training accuracy alone does not.

010 Bias

Meaning. In the bias–variance framework, bias is systematic deviation of the average learned prediction from the true regression function.

$$\mathrm{Bias}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - f^*(x).$$

Notation / context. $f^*(x) = \mathbb{E}[Y \mid X = x]$; expectation is over possible training datasets.
STOP / caveat. This statistical bias differs from a model's intercept b, sampling bias, and demographic unfairness.

011 Variance

Meaning. Sensitivity of a fitted prediction to the particular training sample. A highly flexible model may learn sample-specific noise.

$$\mathrm{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x)) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}\hat{f}_{\mathcal{D}}(x))^2].$$

Notation / context. Keep x fixed; imagine retraining on different samples from the same population.
GO / intuition. More representative data, regularization, and averaging sufficiently different models can reduce variance.

012 Bias–variance decomposition

Meaning. For squared-error regression, expected error separates into systematic error, sample sensitivity, and outcome noise.

$$\mathbb{E}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \mathrm{Bias}(x)^2 + \mathrm{Var}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}(x)) + \sigma^2(x).$$

Notation / context. Assume $Y = f^*(x) + \varepsilon$, with $\mathbb{E}[\varepsilon \mid x] = 0$ and $\mathrm{Var}(\varepsilon \mid x) = \sigma^2(x)$.
STOP / caveat. This is not an unchanged formula for classification accuracy or F1.

013 Overfitting

Meaning. Learning training-specific patterns that do not transfer. A typical warning is falling training loss while validation loss rises.

$$\mathcal{L}_{\text{train}} \downarrow \quad \text{while} \quad \mathcal{L}_{\text{val}} \uparrow .$$

Notation / context. The pattern is diagnostic evidence, not a universal necessary condition. Compare using representative, independent validation data.
GO / intuition. Try regularization, early stopping, simpler models, or more useful data; diagnose before changing architecture.

014 Underfitting

Meaning. Failing to learn useful structure, often producing poor training and validation performance. Causes include weak features, excessive constraints, or incomplete optimization.

$$\text{High training error} \;\Rightarrow\; \text{inspect representation, capacity, and optimization.}$$

STOP / caveat. More epochs help an optimization problem; they cannot make an unsuitable model family represent a missing relationship.
GO / intuition. Compare with a simple baseline and inspect both loss curves before selecting a remedy.

015 L1 and L2 regularization

Meaning. Penalize parameters to constrain the fitted solution. L1 often yields sparse coefficients; L2 smoothly shrinks weights.

$$\mathcal{J}_{L1} = \mathcal{L} + \lambda \sum_j |w_j|, \quad \mathcal{J}_{L2} = \mathcal{L} + \frac{\lambda}{2} \sum_j w_j^2.$$

$$\text{L2 gradient step: } w \leftarrow (1 - \eta\lambda)w - \eta \nabla_w \mathcal{L}.$$

Notation / context. $\lambda \geq 0$: penalty strength; biases are often excluded. This weight-decay equivalence is for ordinary gradient descent.

016 Early stopping

Meaning. Stop training based on held-out performance and restore the best checkpoint. Patience allows several non-improving evaluations before stopping.

$$t^* = \arg\min_t \mathcal{L}_{\text{val}}(\theta_t), \qquad \hat{\theta} = \theta_{t^*}.$$

Notation / context. t: training checkpoint; t^*: best observed checkpoint under the selected metric.
STOP / caveat. The last epoch is not necessarily the best. Do not use test performance as the stopping signal.

017 Data leakage

Meaning. Information enters training or evaluation that would not legitimately be available for the intended prediction. This inflates apparent performance.

Fit preprocessing on $\mathcal{D}_{\text{train}}$; apply it unchanged to held-out data.

Notation / context. Examples: scaling before splitting, future-derived fields, duplicates across splits, or using the target as an input.
GO / intuition. For every feature, ask: could I know this at the exact moment of prediction?

018 Class imbalance

Meaning. Classes occur at unequal frequencies. Accuracy can conceal failure on a rare but important positive class. Weighted losses can alter training emphasis.

$$\mathcal{L} = -\frac{1}{n}\sum_i [\alpha y_i \log p_i + \beta(1-y_i)\log(1-p_i)].$$

Notation / context. α, β: class weights; p_i: predicted positive probability before any needed recalibration.
STOP / caveat. Weighting or resampling may affect calibration. Evaluate with the real deployment prevalence and costs.

019 Confusion matrix

Meaning. Count predictions by both true and predicted class. Fix the positive class before interpreting the table.

	$\hat{y}=1$	$\hat{y}=0$
$y=1$	TP	FN
$y=0$	FP	TN

Notation / context. TP: detected positives; FN: missed positives; FP: false alarms; TN: correctly rejected negatives.
GO / intuition. Build metrics from the question's denominator: all examples, predicted positives, or actual positives.

020 Accuracy

Meaning. Fraction of all examples classified correctly. Appropriate when aggregate correctness reflects the goal and class-specific errors are not hidden.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}.$$

Notation / context. If only 1 percent of examples are positive, predicting every example negative gives 99 percent accuracy and zero recall.
STOP / caveat. High accuracy does not establish useful positive detection.

021 Precision

Meaning. Among predicted positives, the fraction actually positive. It answers how trustworthy positive flags are.

$$P = \frac{TP}{TP+FP}.$$

Notation / context. Example: 8 true fraud cases among 10 flagged transactions gives $P = 0.8$. The denominator is predicted positives.
STOP / caveat. Precision depends on prevalence and the decision rule; define a convention when there are no positive predictions.

022 Recall

Meaning. Among actual positives, the fraction detected. Also called sensitivity or true-positive rate.

$$R = TPR = \frac{TP}{TP + FN}.$$

Notation / context. Example: detecting 8 of 20 actual fraud cases gives $R = 0.4$. The denominator is actual positives.
GO / intuition. Prioritize recall when missing positives is costly, while still accounting for the cost of false alarms.

023 F1 score

Meaning. Harmonic mean of precision and recall; low when either is low. It ignores true negatives.

$$F_1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN}.$$

Notation / context. P: precision; R: recall. More generally, $F_\beta = (1 + \beta^2)PR/(\beta^2 P + R)$; $\beta > 1$ emphasizes recall.
STOP / caveat. F1 is not a direct encoding of arbitrary business costs and depends on a chosen threshold.

024 Decision threshold

Meaning. Convert a score or probability into a class. Lowering the threshold generally flags more examples, increasing both true- and false-positive counts.

$$\hat{y}_i = \mathbf{1}[p_i \geq \tau].$$

Notation / context. τ: threshold. For calibrated probabilities and false-positive cost c_{FP} versus false-negative cost c_{FN}, a simple cost-optimal threshold is $c_{FP}/(c_{FP} + c_{FN})$.
STOP / caveat. That cost formula assumes no additional decision costs or constraints.

025 ROC curve and ROC-AUC

Meaning. ROC plots true-positive rate against false-positive rate as the threshold varies. AUC summarizes score discrimination.

$$FPR = \frac{FP}{FP + TN}, \quad AUC = P(s^+ > s^-) + \tfrac{1}{2}P(s^+ = s^-).$$

Notation / context. s^+, s^-: scores of randomly drawn positive and negative examples. The upper-left corner is perfect separation at a threshold.
STOP / caveat. AUC does not measure calibration; a low FPR can still create many false alarms among numerous negatives.

026 Precision–recall curve

Meaning. Plot precision against recall across thresholds. Particularly useful when positives are rare and the quality of positive results matters.

$$\pi = P(Y = 1), \qquad \text{population precision of random selection} = \pi.$$

Notation / context. π: positive prevalence. Average precision summarizes precision increments weighted by recall gains; it need not equal trapezoidal PR area.
STOP / caveat. Compare PR metrics with attention to prevalence, candidate selection, and interpolation convention.

027 Calibration

Meaning. Predicted probabilities agree with observed frequencies. Among predictions near 0.8, approximately 80 percent should be positive.

$$P(Y = 1 \mid \hat{p} = p) = p, \qquad \hat{p}_{\text{Platt}} = \sigma(as + b).$$

Notation / context. s: original score; a, b: fitted calibration parameters. Isotonic regression instead learns a nondecreasing mapping $g(s)$.
STOP / caveat. Fit calibrators on held-out predictions. Good ranking does not imply calibrated probabilities.

028 Linear regression

Meaning. Predict a numerical target using a weighted sum of features plus an intercept. Fit by minimizing squared residuals.

$$\hat{y}_i = w^\top x_i + b, \quad \mathcal{L} = \frac{1}{2n}\sum_i (\hat{y}_i - y_i)^2.$$

$$\nabla_w \mathcal{L} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)x_i, \quad \partial_b \mathcal{L} = \frac{1}{n}\sum_i (\hat{y}_i - y_i).$$

GO / intuition. Update weights opposite these gradients. "Linear" means linear in coefficients; engineered nonlinear features are allowed.

029 Logistic regression

Meaning. Model a binary probability by applying sigmoid to a linear logit. Fit with binary cross-entropy.

$$z_i = w^\top x_i + b, \quad p_i = \sigma(z_i), \quad \log\frac{p_i}{1-p_i} = z_i.$$

$$\mathcal{L} = -\frac{1}{n}\sum_i [y_i \log p_i + (1-y_i)\log(1-p_i)], \quad \nabla_w \mathcal{L} = \frac{1}{n}\sum_i (p_i - y_i)x_i.$$

Notation / context. $y_i \in \{0, 1\}$; the log odds are linear in the supplied features.

030 Sigmoid

Meaning. Map a real-valued logit to a number between zero and one. Used for binary probabilities, independent labels, and gates.

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1-\sigma(z)).$$

Notation / context. $\sigma(0) = 0.5$; large positive inputs approach 1, large negative inputs approach 0.
STOP / caveat. A sigmoid output is not automatically calibrated. Saturated regions have small derivatives, which can weaken gradient flow.

031 Cross-entropy, likelihood, and KL

Meaning. Negative log-likelihood penalizes assigning low probability to observed outcomes. For independent training examples, maximizing likelihood equals minimizing summed negative log-probability.

$$\mathcal{L} = -\frac{1}{n}\sum_i \log p_\theta(y_i \mid x_i), \quad H(P,Q) = -\sum_x P(x)\log Q(x).$$

$$D_{KL}(P\|Q) = \sum_x P(x)\log\frac{P(x)}{Q(x)}, \quad H(P,Q) = H(P) + D_{KL}(P\|Q).$$

GO / intuition. For fixed target distribution P, minimizing cross-entropy over Q minimizes this KL divergence.

032 Decision tree

Meaning. Recursively split examples by feature tests. Choose splits greedily to reduce weighted child impurity; leaves store a class distribution or numerical prediction.

$$\Delta I = I_{\text{parent}} - \frac{n_L}{n} I_L - \frac{n_R}{n} I_R.$$

Notation / context. I: impurity; n_L, n_R: child sample counts. Regression splits often minimize within-leaf squared error.
STOP / caveat. Training a standard tree is greedy, not globally optimal. Unrestricted depth can overfit.

033 Gini impurity and entropy

Meaning. Quantify how mixed a classification node is. Both equal zero for a pure node.

$$G = 1 - \sum_{k=1}^{K} p_k^2, \qquad H = -\sum_{k=1}^{K} p_k \log p_k.$$

Notation / context. p_k: class-k fraction in the node; use the convention $0 \log 0 = 0$. Binary Gini is largest at $p = 0.5$.
GO / intuition. A useful split reduces weighted impurity. These are split criteria, not the whole tree's final evaluation metric.

034 Bagging and random forest

Meaning. Bagging fits models to bootstrap samples and aggregates predictions. Random forests additionally randomize candidate features at splits to reduce tree correlation.

$$\hat{f}(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x), \quad \mathrm{Var}(\bar{e}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Notation / context. B: models; the variance formula assumes errors with equal variance σ^2 and equal pairwise correlation ρ.
GO / intuition. Averaging helps most when model errors are not highly correlated.

035 Gradient boosting

Meaning. Build an additive predictor sequentially. Fit each new learner to negative gradients of the current prediction loss.

$$r_{im} = -\left.\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)}\right|_{F=F_{m-1}}, \quad F_m = F_{m-1} + \eta\gamma_m h_m.$$

Notation / context. m: boosting round; h_m: new learner; η: shrinkage; γ_m: optional fitted step. For $\ell = \frac{1}{2}(y-F)^2$, $r_{im} = y_i - F_{m-1}(x_i)$.
STOP / caveat. "Fit the residuals" is exact for squared loss, not every boosting loss.

036 ★ Support vector machine

Meaning. Find a large-margin separating boundary. Traditional binary SVM labels are $y_i \in \{-1, +1\}$, unlike logistic regression's usual 0/1 convention.

$$f(x) = w^\top x + b, \quad \hat{y} = \operatorname{sign} f(x), \quad \min_{w,b} \frac{1}{2}\|w\|^2 \text{ s.t. } y_i f(x_i) \geq 1.$$

Notation / context. The displayed hard-margin formulation requires separability. Soft margin minimizes $\frac{1}{2}\|w\|^2 + C\sum_i \max(0, 1 - y_i f(x_i))$.
STOP / caveat. The SVM score is not inherently a probability.

037 ★ Margin and support vectors

Meaning. A point's geometric distance from the hyperplane is its absolute score divided by the weight norm. Support vectors determine the fitted boundary.

$$\operatorname{dist}(x, \mathcal{H}) = \frac{|w^\top x + b|}{\|w\|}, \quad \text{margin width} = \frac{2}{\|w\|}.$$

Notation / context. Width assumes supporting planes $w^\top x + b = \pm 1$. In the dual, support vectors have $\alpha_i > 0$; soft-margin support vectors may lie inside the margin or be misclassified.

038 ★ Kernel trick

Meaning. Compute inner products in an implicit feature space without constructing the transformed vectors explicitly.

$$K(x, z) = \phi(x)^\top \phi(z), \quad f(x) = \sum_i \alpha_i y_i K(x_i, x) + b.$$

Notation / context. K: kernel function; $K_{ij} = K(x_i, x_j)$: Gram matrix entry; ϕ: feature map; α_i: learned dual coefficient.
GO / intuition. The boundary is linear in feature space but can be nonlinear in original inputs. RBF: $K(x, z) = e^{-\gamma \|x - z\|^2}$.

039 ★ SVM training: C, gamma, and alpha

Meaning. Soft-margin training balances weight norm against violations ξ_i. The primal and kernel dual are:

$$\min_{w, b, \xi \geq 0} \frac{1}{2}\|w\|^2 + C \sum_i \xi_i \quad \text{s.t. } y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

$$\max_\alpha \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij}, \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0.$$

Notation / context. ξ_i: slack for margin violation; C: penalty/bound; γ: RBF locality, not a learning rate. Larger C penalizes violations more; larger γ gives more local influence.
STOP / caveat. Scale features; neither increasing C nor γ guarantees better generalization.

040 Distribution shift and concept drift

Meaning. Deployment and training distributions may differ in inputs, labels, or the input–outcome relationship.

$$P(X,Y) = P(Y \mid X)P(X) = P(X \mid Y)P(Y).$$

Notation / context. Covariate shift: $P(X)$ changes with stable $P(Y \mid X)$. Label shift: $P(Y)$ changes with stable $P(X \mid Y)$. Concept drift: $P(Y \mid X)$ changes.
GO / intuition. Retrain if relationships shift. Add features if signals are missing. Change architecture only if the current one can't capture the new patterns.

041 Object-oriented design

Meaning. Choose objects, responsibilities, interfaces, ownership, and collaborations before and during implementation. Design describes who should do what and which dependencies should exist.

`Recommender` uses `Encoder` and `Ranker`.

Notation / context. For your fashion system, decide that `Look` owns item metadata while `Recommender` combines candidate retrieval and scoring.
GO / intuition. Explain responsibility boundaries and how a likely change stays localized. Design is more than naming classes.

042 Object-oriented programming

Meaning. Implement behavior using objects that combine state and operations. Classes, methods, composition, inheritance, and polymorphism are tools for realizing a design.

```
model = Recommender(encoder, ranker)
```

Notation / context. OOD chooses responsibilities and relationships; OOP writes and executes the classes and objects that express them.
STOP / caveat. Using many classes does not guarantee a good design; prefer a clear implementation over unnecessary abstraction.

043 Class versus object

Meaning. A class defines a type's structure and behavior; an object is one instance with its own identity and state.

```
u1 = UserProfile(...)    u2 = UserProfile(...)
```

Notation / context. Both objects have the same class but can hold different preferences. In Python, equality `==` and identity `is` ask different questions.
STOP / caveat. Mutable class attributes can be shared by all instances; initialize per-instance mutable state on the instance.

044 Attribute versus method

Meaning. An attribute is a named value associated with an object or class. A method is callable behavior obtained through the object or class.

```
look.color    look.describe()
```

Notation / context. Methods may read or update attributes. Python properties can expose computed or validated attribute-style access.
GO / intuition. Ask whether the operation belongs with the state it needs. Keep responsibilities cohesive rather than scattering state changes.

045 Object creation and initialization

Meaning. Creating an object allocates or returns an instance; initialization establishes its starting state and invariants. In Python, these roles are principally `__new__` and `__init__`.

$$\{\text{valid arguments}\} \quad \texttt{Account(balance)} \quad \{balance \geq 0\}.$$

Notation / context. Interview discussion often calls `__init__` the constructor informally. It should not return a non-`None` value.

046 Encapsulation

Meaning. Keep state and related behavior together, controlling changes through operations that preserve invariants.

$$\{B \geq a \geq 0\} \quad \texttt{withdraw}(a) \quad \{B' = B - a \geq 0\}.$$

Notation / context. B: current balance; a: withdrawal; B': resulting balance. An operation should enforce the contract rather than expose arbitrary balance edits.
STOP / caveat. Python's underscore naming and name mangling are not security boundaries.

047 Abstraction

Meaning. Expose a useful contract while hiding implementation choices. Callers depend on promised behavior rather than internal data structures or algorithms.

$$\texttt{predict(features)} \longrightarrow \texttt{prediction}.$$

Notation / context. The same interface may hide a tree, a linear model, or a neural network. Inputs, outputs, exceptions, and behavioral guarantees form the contract.
GO / intuition. A good abstraction removes irrelevant detail without concealing assumptions the caller needs.

048 Inheritance

Meaning. Create a subtype from a base class, reusing or specializing behavior. A valid subtype should honor the behavioral promises of the base type.

$$\texttt{Dog} \subseteq \texttt{Animal} \quad \text{as a behavioral "is-a" relationship.}$$

Notation / context. The substitutability principle means code expecting the base type should still behave correctly with the subtype.
STOP / caveat. Reuse alone is not a sufficient reason for inheritance; deep hierarchies can create tight coupling.

049 Composition

Meaning. Build an object from collaborating objects and delegate tasks to them. This expresses a "has-a" or "uses-a" relationship.

`Recommender(encoder, ranker)`

Notation / context. The recommender contains dependencies rather than inheriting from each one. Different encoders can be substituted through the same contract.
GO / intuition. Composition often makes changes and tests easier because dependencies can be replaced independently.

050 Polymorphism

Meaning. Use different types through a common expected interface. The same call can invoke different implementations.

`model.predict(x)` for multiple compatible model types.

Notation / context. Python's duck typing permits this without a shared explicit superclass. Correct signatures alone do not establish compatible semantics.
GO / intuition. Code against the behavior required, then substitute any implementation that honors that contract.

051 Interface and protocol

Meaning. Describe the operations, inputs, outputs, and behavior a component promises. An implementation supplies the mechanism.

`Encoder.encode(item)` $\longrightarrow \mathbb{R}^d$.

Notation / context. Python protocols express structural compatibility; abstract base classes can define explicit inheritance-based interfaces. Dimensions and normalization may be part of an encoder contract.
STOP / caveat. Matching method names is insufficient if units, shapes, error behavior, or meaning differ.

052 Single-responsibility principle

Meaning. A component should have one coherent responsibility or main reason to change. Separate unrelated policies so changing one does not destabilize the others.

load data | predict | format report.

Notation / context. These may deserve separate components if they change for different reasons.
STOP / caveat. This does not require one method per class. Tiny fragmented classes can obscure rather than clarify behavior.

053 Cohesion versus coupling

Meaning. Cohesion measures how closely responsibilities within a component belong together. Coupling concerns dependencies between components. Prefer coherent internals and controlled external dependencies.

Aim: high cohesion and manageable coupling.

Notation / context. A ranker that also sends email has unrelated responsibilities. A ranker that relies on a narrow encoder interface has a controlled dependency.
GO / intuition. Evaluate a design by tracing how far a realistic requirement change would spread.

054 Dependency injection

Meaning. Provide a component's dependencies externally instead of having it construct fixed dependencies internally.

```
service = Service(database=fake_db)
```

Notation / context. Constructor injection makes dependencies explicit. Tests can supply a fake database; production supplies a real client.
GO / intuition. Depend on the required contract, not a particular concrete implementation. Avoid adding injection machinery where a simple argument suffices.

055 API

Meaning. An application programming interface defines how software components interact. It may be a function signature, class interface, or network service.

$$f : \mathcal{X} \to \mathcal{Y} \quad \text{with specified validation and failure behavior.}$$

Notation / context. A prediction API should define feature schema, output meaning, model/version semantics when relevant, and possible errors.
STOP / caveat. An API is not necessarily an HTTP endpoint; local libraries also have APIs.

056 HTTP API and endpoint

Meaning. An HTTP API exposes operations over HTTP. An endpoint combines a route and supported method, with a request/response contract.

$$\texttt{POST /predict} \quad + \quad \text{JSON input} \quad \longrightarrow \quad \text{status} + \text{JSON output.}$$

Notation / context. Responses should distinguish successful predictions, invalid inputs, authorization failures, and service failures.
GO / intuition. Discuss validation, error handling, latency, and compatibility when asked to design an endpoint.

057 HTTP methods

Meaning. GET retrieves a representation; POST submits data or creates resources; PUT replaces a representation; PATCH partially modifies; DELETE removes a resource.

Method semantics guide caching, retries, and client expectations.

Notation / context. GET is intended to be safe and idempotent. PUT and DELETE are idempotent by intended effect; POST and PATCH are not inherently idempotent.
STOP / caveat. A method name does not force an implementation to follow HTTP semantics.

058 Idempotency

Meaning. Repeating an operation has the same intended state effect as performing it once. This supports safe retry behavior.

$$f(f(s)) = f(s).$$

Notation / context. s: system state; f: fixed operation. Setting a preference to dark is idempotent; incrementing a counter usually is not.
STOP / caveat. Responses can differ across repeated requests even when the intended state effect is idempotent.

059 Exception handling

Meaning. Respond to exceptional conditions where recovery or useful context is possible. Clean up resources even when operations fail.

`try` $\rightarrow$ `except` and/or `finally`.

Notation / context. Catch specific exceptions when possible. Use context managers for resources and distinguish invalid user input from internal bugs.
STOP / caveat. Silently swallowing broad exceptions can convert visible failures into misleading output or corrupted state.

060 Mutability and aliasing

Meaning. A mutable object can change after creation. Aliasing means multiple references point to the same object. A shallow copy duplicates only the outer container.

`b = a` $\Rightarrow$ both names reference the same object.

Notation / context. Mutating a shared list through either reference affects that list. Nested mutable objects remain shared after a shallow copy.
STOP / caveat. A mutable default argument is created once, so its state can persist across calls.

061 Unit testing

Meaning. Test a small unit's observable behavior with controlled inputs and expected outcomes. Isolate unrelated dependencies when that improves clarity.

$$\text{Arrange} \rightarrow \text{Act} \rightarrow \text{Assert}.$$

Notation / context. Example: test that a ranking function returns expected order for a tiny known score vector.
GO / intuition. Test the contract and meaningful failure modes; tests tied only to implementation details can break during harmless refactors.

062 Integration testing

Meaning. Test whether multiple real components work together correctly. It catches interface, schema, configuration, and interaction problems missed by isolated tests.

$$\text{HTTP handler} \rightarrow \text{preprocessor} \rightarrow \text{model}.$$

Notation / context. Example: verify a serialized feature vector reaches the model with the intended shape, types, and ordering.
STOP / caveat. Passing unit tests does not prove independently developed components agree on their shared contract.

063 End-to-end testing

Meaning. Exercise a complete user or system workflow through its major layers. It provides broad confidence but failures can be harder to localize.

$$\text{request} \rightarrow \text{service} \rightarrow \text{storage/model} \rightarrow \text{response}.$$

Notation / context. Example: submit a recommendation request and check the returned items satisfy the requested constraints.
GO / intuition. Use a small set of valuable workflows alongside faster unit and integration tests.

064 Test case

Meaning. A particular setup, action/input, and expected outcome. A test must identify what would make it pass or fail.

$$\text{input } x \quad \longrightarrow \quad \text{expected } f(x).$$

Notation / context. For sorting, use a known input with duplicates and verify both order and preservation of elements.
STOP / caveat. Printing output and visually glancing at it is weaker than asserting the specified behavior.

065 Edge case

Meaning. A case at or near a boundary of valid behavior: empty input, one element, zero capacity, maximum size, or a target at an endpoint.

$$n \in \{0, 1\}, \quad c = 0, \quad \text{target at index 0 or } n - 1.$$

Notation / context. Select boundaries from the actual problem constraints; an invalid input is not necessarily a required valid edge case.
GO / intuition. Ask which branch, index, or initialization behaves differently at the boundary.

066 Corner case

Meaning. An unusual interaction between multiple conditions. The distinction from edge case is informal; interviewers may use the terms interchangeably.

$$\text{single isolated source} \quad + \quad \text{unreachable target}.$$

Notation / context. Other examples include repeated values at a search boundary or an empty component inside an otherwise nonempty structure.
GO / intuition. Combine boundary conditions rather than testing each only in isolation.

067 Boundary-value analysis

Meaning. Test values at and immediately around points where behavior changes. This targets comparison and off-by-one mistakes.

$$\text{Allowed } 18 \le a \le 65: \quad a \in \{17, 18, 19, 64, 65, 66\}.$$

Notation / context. For a threshold τ, test below, equal to, and above τ. Include invalid boundaries when validation is part of the contract.
STOP / caveat. A condition using $<$ can behave differently from one using $\le$ at exactly the boundary.

068 Equivalence partitioning

Meaning. Divide inputs into groups expected to trigger similar behavior, then test representatives from each group.

$$\mathbb{R} = (-\infty, 0) \cup \{0\} \cup (0, \infty).$$

Notation / context. These are natural partitions for a sign function. Additional partitions may be needed for invalid types, missing values, or numerical extremes.
GO / intuition. Combine partitioning with boundary tests for systematic coverage without exhaustive enumeration.

069 Assertion

Meaning. An executable check that a condition holds. Tests use assertions to report a failure when observed behavior violates expectations.

```
assert actual == expected.
```

Notation / context. For floating-point results, use a suitable absolute/relative tolerance rather than requiring exact equality.
STOP / caveat. Python assertions can be disabled; do not rely on them for essential production input validation or authorization.

070 Test oracle

Meaning. The source of truth used to decide an expected result. It may be a known example, mathematical property, or slow reference implementation.

$$\texttt{optimized(x)} \stackrel{?}{=} \texttt{brute_force(x)}.$$

Notation / context. Use small randomly generated inputs when the brute-force oracle is expensive. Ensure the reference is independently understandable.
STOP / caveat. If the expected result repeats the same bug as the implementation, a passing test provides little confidence.

071 Property-based testing

Meaning. Generate many inputs and check general properties instead of only predetermined examples.

$$\text{sort}(x) \text{ is ordered}, \quad \text{multiset}(\text{sort}(x)) = \text{multiset}(x).$$

Notation / context. A framework may shrink a failing example to a simpler counterexample. Properties must follow from the contract.
GO / intuition. For a queue, compare random operation sequences against a simple reference model.

072 Mock, stub, and fake

Meaning. Test doubles replace dependencies. A stub returns predetermined responses; a fake is a simplified working implementation; a mock commonly verifies interactions.

$$\text{real dependency} \longrightarrow \text{controlled test substitute}.$$

Notation / context. Terminology varies. Use a fake in-memory repository for storage behavior or a mock to verify a required notification call.
STOP / caveat. Over-mocking can make tests confirm calls while missing broken real integration.

073 Regression testing

Meaning. Check that changes preserve previously correct behavior. A bug fix should often include a case reproducing the failure.

$$\text{buggy version: fail} \quad \longrightarrow \quad \text{fixed version: pass.}$$

Notation / context. The regression test should describe the intended behavior and remain meaningful after refactoring.
GO / intuition. Retain a minimal failing example rather than only a huge input whose cause is difficult to understand.

074 Invariant

Meaning. A property that remains true at a specified point during an algorithm or object's lifetime. Invariants support correctness proofs and test design.

Binary-search invariant: any possible target lies in the retained interval.

Notation / context. A loop proof shows initialization, preservation, and useful consequences at termination. For a min-heap, each parent is no larger than its children.
GO / intuition. State the invariant before explaining why each update is safe.

075 Preconditions and postconditions

Meaning. Preconditions describe required starting assumptions; postconditions describe guaranteed results when those assumptions hold.

$$\{P\} \quad C \quad \{Q\}.$$

Notation / context. P: precondition; C: operation; Q: postcondition. Binary search requires sorted input and returns an agreed matching index or not-found result.
STOP / caveat. If callers can violate a precondition, define whether the implementation validates, rejects, or assumes it.

YELLOW STOCK Algorithms, sorting, searching, and dynamic programming

Cards 76–95. Read the local notation before memorizing the equations.

n, m	Input sizes; define what each counts before analyzing loops.
$T(n),\ S(n)$	Runtime and auxiliary-space growth. State the computational model and case.
$O,\ \Omega,\ \Theta$	Asymptotic upper bound, lower bound, and tight bound.
$T(n) = aT(n/b) + f(n)$	Divide-and-conquer recurrence: a subproblems, size n/b, nonrecursive work $f(n)$.
ℓ, r, mid	Left boundary, right boundary, midpoint. Specify inclusive/exclusive interval conventions.
$DP[i, c],\ w_i,\ v_i,\ W$	Knapsack state, item weight, item value, total capacity. Here W is not a neural weight matrix.
$h,\ k$	Tree height and retained item count or key-range size, depending on the card.
$\lvert x\rvert,\ \varnothing,\ \infty$	Sequence length, empty set, and an unreachable/initialization sentinel.

GO / intuition. For every algorithm: define state, invariant/recurrence, base cases, order, answer, and complexity.

STOP / caveat. Big-O is a bound, not a synonym for worst case. A nested loop is not automatically quadratic.

076 Big-O, Big-Theta, and Big-Omega

Meaning. Describe asymptotic growth, ignoring constant factors and sufficiently small inputs. For nonnegative functions:

$$f \in O(g) \iff \exists c, n_0 > 0 : \ f(n) \le cg(n) \text{ for all } n \ge n_0.$$

Notation / context. Ω reverses the inequality for a lower bound; Θ requires both bounds. $3n + 7 \in \Theta(n)$.
STOP / caveat. Specify worst, average, expected, or amortized case separately. An $O(n^2)$ upper bound can be true but loose for a linear algorithm.

077 Time versus space complexity

Meaning. Time describes work as input size grows; space describes memory. Clarify whether memory includes inputs and outputs or only auxiliary storage.

$$\text{Recursive auxiliary space} \approx \text{maximum active depth} \times \text{space per frame}.$$

Notation / context. A balanced recursive tree traversal uses $O(h)$ call-stack space, not one simultaneously active frame for every node.
STOP / caveat. Output storage can dominate: generating many solutions costs at least the time needed to write them.

078 Worst, average, and amortized analysis

Meaning. Worst case bounds the most expensive size-n input. Average case assumes an input distribution. Amortized analysis bounds a whole operation sequence without assuming random inputs.

$$\text{amortized cost} = \frac{\text{total cost of a sequence}}{\text{number of operations}}.$$

Notation / context. Dynamic-array append is amortized $O(1)$ even though a resizing append is $O(n)$.
STOP / caveat. Amortized constant time does not mean every operation has constant worst-case latency.

079 ★ Comparison sorting lower bound

Meaning. Sorting arbitrary distinct keys by comparisons requires enough decisions to distinguish all $n!$ orderings.

$$2^h \geq n! \quad \Rightarrow \quad h \geq \log_2(n!) = \Omega(n \log n).$$

Notation / context. h: decision-tree height, corresponding to worst-case comparisons. Merge sort achieves the matching asymptotic upper bound.
STOP / caveat. Counting and radix sorts exploit key structure, so this comparison-only lower bound does not prohibit faster runtimes for them.

080 ★ Stable and in-place sorting

Meaning. Stability preserves original relative order among equal-key elements. In-place sorting uses little extra storage, with conventions differing on recursion-stack accounting.

$$a_i.key = a_j.key,\ i < j \quad \Rightarrow \quad a_i \text{ remains before } a_j \text{ in a stable sort.}$$

Notation / context. Stable sorts support sorting by a secondary key first, then a primary key. Stability and memory use are independent properties.
STOP / caveat. An array being modified in place does not imply zero auxiliary memory.

081 ★ Merge sort

Meaning. Split into halves, recursively sort both, then merge the sorted sequences using advancing pointers.

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

Notation / context. Standard array implementation: $O(n)$ auxiliary storage; base case $n \leq 1$. Choose the left item first on equality to preserve stability.
GO / intuition. There are about $\log_2 n$ levels, each doing total linear merge work. This is divide and conquer.

082 ★ Quicksort

Meaning. Partition around a pivot, then recursively sort the resulting sides. Runtime depends on partition balance.

$$T(n) = T(k) + T(n-k-1) + \Theta(n).$$

Notation / context. Balanced or randomized-expected behavior: $O(n \log n)$. Repeated extreme pivots: $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$.
STOP / caveat. Typical in-place quicksort is unstable; stack depth can be $O(n)$ without safeguards. Equal keys need thoughtful partition handling.

083 ★ Heapsort

Meaning. Build a max-heap, swap its root into the final unsorted position, reduce heap size, and restore heap order. Repeat.

$$T(n) = O(n) + \sum_{m=2}^{n} O(\log m) = O(n \log n).$$

Notation / context. Standard array heapsort has $O(1)$ auxiliary space and $O(n \log n)$ worst-case time. It is generally unstable.
GO / intuition. The heap keeps the next extreme accessible without maintaining a fully sorted interior.

084 ★ Insertion sort

Meaning. Maintain a sorted prefix. Insert each next element by shifting larger preceding elements to the right.

$$\text{Worst work} = \sum_{i=1}^{n-1} \Theta(i) = \Theta(n^2).$$

Notation / context. Already sorted input takes $\Theta(n)$ with the usual implementation. Standard insertion sort is in-place and stable when equal keys are not moved past each other.
GO / intuition. Useful for small or nearly sorted inputs; its work is related to the number of inversions.

085 ★ Counting sort

Meaning. Count integer-key occurrences over a bounded range and use those counts to construct ordered output.

$$T(n,k) = \Theta(n+k).$$

Notation / context. n: item count; k: number of possible key values in the represented range. A standard stable output-array version uses $O(n+k)$ auxiliary space.
STOP / caveat. A huge key range can make counting sort impractical. Stability requires placing complete records carefully, not merely emitting key counts.

086 Binary search

Meaning. Repeatedly discard half of a sorted random-access search space. One valid convention retains a half-open interval $[\ell, r)$.

$$mid = \ell + \lfloor (r-\ell)/2 \rfloor, \qquad T(n) = T(n/2) + O(1) = O(\log n).$$

Notation / context. For lower bound: if $a[mid] < target$, set $\ell = mid + 1$; otherwise set $r = mid$. Stop when $\ell = r$.
STOP / caveat. Returning a matching element also requires checking bounds and equality after the lower-bound search.

087 ★ Divide and conquer

Meaning. Break a problem into smaller subproblems, solve them recursively, and combine results.

$$T(n) = aT(n/b) + f(n).$$

Notation / context. a: number of recursive subproblems; n/b: their size; $f(n)$: splitting and combining work. Merge sort has $a = b = 2$ and $f(n) = \Theta(n)$.
GO / intuition. Draw the recursion tree and total work per level. Unlike common DP cases, subproblems need not overlap.

088 ★ Recurrences and Master theorem

Meaning. For suitable recurrences $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$.

$$f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log n).$$

Notation / context. Polynomially smaller f gives $\Theta(n^{\log_b a})$; polynomially larger f gives $\Theta(f(n))$ under the regularity condition.
STOP / caveat. The theorem has conditions and does not directly cover $T(n) = T(n-1) + n$ or arbitrary uneven subproblems.

089 Recursion

Meaning. A function solves a state using smaller states, with a base case that ends the descent. Count the entire call tree, not only depth.

$$fact(n) = \begin{cases} 1 & n = 0 \\ n\, fact(n-1) & n > 0. \end{cases}$$

Notation / context. Factorial uses $O(n)$ recursive calls and $O(n)$ stack space under constant-cost arithmetic.
STOP / caveat. Depth describes simultaneous frames; total nodes describe total calls. These can differ dramatically in branching recursion.

090 Backtracking

Meaning. Explore a decision, recurse, undo it, and try another. Prune prefixes that cannot lead to a valid solution.

$$\text{Balanced-parenthesis prefix: } 0 \le close \le open \le n.$$

Notation / context. Add an opening parenthesis if $open < n$; add a closing one if $close < open$. Output when both equal n.
GO / intuition. For n pairs, there are $C_n = \frac{1}{n+1}\binom{2n}{n}$ outputs; writing all strings costs at least $\Omega(nC_n)$.

091 Dynamic programming

Meaning. Reuse well-defined subproblem answers. State must contain enough information for transitions; evaluate states after their dependencies.

$$DP[s] = \text{best}_{a \in A(s)}\{\text{local contribution} + DP[\text{next}(s, a)]\}.$$

Notation / context. This schematic equation fits many optimization DPs; counting DPs use sums instead of best.
GO / intuition. Write state, recurrence, base cases, dependency order, final answer, and number-of-states times work-per-state.

092 Memoization versus tabulation

Meaning. Memoization computes requested states recursively and caches results. Tabulation explicitly computes states in a valid order.

$$F_n = F_{n-1} + F_{n-2}, \quad F_0 = 0, \quad F_1 = 1.$$

Notation / context. Fibonacci has $O(n)$ states and $O(1)$ work per state. Memoization uses a cache plus stack; tabulation can keep only two preceding values.
STOP / caveat. Caching helps only when states repeat and cache keys fully describe the subproblem.

093 Greedy algorithms

Meaning. Commit to a locally preferred choice without exploring every alternative. Correctness requires a property such as an exchange argument.

Fractional-knapsack priority: $r_i = v_i/w_i$.

Notation / context. Taking the highest value-per-weight first is optimal when fractions are allowed, but not generally for 0/1 knapsack.
STOP / caveat. A convincing local rule is not proof. Search for a counterexample, then explain why no counterexample exists when claiming optimality.

094 0/1 knapsack

Meaning. Choose each item at most once to maximize value under integer capacity W. Let $DP[i, c]$ use only the first i items.

$$DP[i,c] = \begin{cases} DP[i-1,c] & w_i > c \\ \max(DP[i-1,c], v_i + DP[i-1, c-w_i]) & w_i \le c. \end{cases}$$

Notation / context. $DP[0, c] = 0$; positive weights give $DP[i, 0] = 0$. Time $O(nW)$; rolling storage $O(W)$.
STOP / caveat. For one-dimensional 0/1 DP, update capacities downward to avoid reusing the current item.

095 P, NP, NP-hard, NP-complete

Meaning. P: polynomial-time solvable decision problems. NP: decision problems with polynomially verifiable certificates. NP-hard: at least as hard as every NP problem under the specified reductions. NP-complete: both NP and NP-hard.

$$NP\text{-complete} = NP \cap NP\text{-hard}.$$

Notation / context. Decision knapsack and decision traveling salesman are NP-complete; optimization versions are NP-hard. Knapsack's $O(nW)$ DP is pseudopolynomial because W needs only $O(\log W)$ input bits.
STOP / caveat. NP does not mean "not polynomial"; whether $P = NP$ remains unresolved.

BLUE STOCK Graphs, graph complexity, Dijkstra, and A*

Cards 96–110. Read the local notation before memorizing the equations.

$G = (V, E),\ \lvert V\rvert,\ \lvert E\rvert$	Graph, vertex set, edge set, and their sizes. Runtime formulas often abbreviate the sizes as V, E.
$u, v,\ s, t$	Vertices; source and target. Here t is not a training step.
$w(u, v)$	Edge weight/cost; unrelated to a learned regression coefficient.
$d[v],\ \pi[v]$	Best tentative source distance and predecessor for reconstructing a path.
$\delta(s, v)$	True optimal source-to-vertex distance.
$g(n),\ h(n),\ f(n)$	A* cost so far, heuristic remaining cost, total priority $f = g + h$. Here n is a node.
$h^*(n)$	True optimal remaining cost to the goal.
b, d	In search-tree complexity, branching factor and solution depth; neither is a model bias here.

GO / intuition. First ask: directed? weighted? negative edges? sparse? one source? one goal? These assumptions choose the algorithm.

STOP / caveat. BFS minimizes edge count; Dijkstra needs nonnegative edge weights; A* guarantees depend on heuristic and reopening policy.

096 Graph terminology

Meaning. A graph is a set of vertices and edges representing relationships. Edges may be directed, undirected, weighted, or unweighted.

$$G = (V, E), \qquad w : E \to \mathbb{R}.$$

Notation / context. A path is a sequence of connected vertices; its weighted cost is the sum of its edge costs. Degree counts incident edges, with in/out degrees for directed graphs.
GO / intuition. Before coding, identify graph assumptions and choose a representation.

097 ★ Adjacency list

Meaning. Store each vertex's outgoing neighbors, with weights if needed. Undirected edges generally appear in both endpoint lists.

$$S = O(|V| + |E|), \qquad \text{scan neighbors of } u: \ O(\deg(u)).$$

Notation / context. Across a full traversal, directed adjacency entries total $|E|$; undirected entries total $2|E|$. Constant factor two does not change Big-O.
GO / intuition. Usually effective for sparse graphs. Edge-existence checks may require scanning a list unless using an auxiliary set/map.

098 ★ Adjacency matrix

Meaning. Use a table indexed by vertex pairs to indicate edge existence or weight.

$$A \in \mathbb{R}^{|V| \times |V|}, \quad S = \Theta(|V|^2).$$

Notation / context. Checking one pair is $O(1)$; scanning all possible neighbors of a vertex is $O(|V|)$. Use a clear sentinel for absent weighted edges.
STOP / caveat. Zero can be a valid edge weight, so using zero to mean "no edge" may be ambiguous.

099 ★ Breadth-first search

Meaning. Explore vertices by increasing number of edges from a source, using a FIFO queue and visited tracking.

$$d[s] = 0, \quad d[v] = d[u] + 1 \text{ on first discovery from } u.$$

Notation / context. With adjacency lists, full traversal takes $O(|V| + |E|)$ time and $O(|V|)$ auxiliary space. Mark visited when enqueuing to avoid duplicates.
STOP / caveat. BFS finds minimum-edge paths, not general minimum-weight paths. Equal edge costs make the objectives coincide.

100 ★ Depth-first search

Meaning. Explore one branch fully before backtracking, using recursion or an explicit LIFO stack. Useful for reachability, components, cycles, and traversal orders.

$$T = O(|V| + |E|), \qquad S_{\text{aux}} = O(|V|) \quad \text{for adjacency lists.}$$

Notation / context. Visited tracking prevents endless cycles. Recursive depth can reach $|V|$ in a chain.
STOP / caveat. An arbitrary DFS discovery path is not generally a shortest path. Directed cycle detection needs more than one undifferentiated visited flag.

101 ★ Runtime analysis of graphs

Meaning. Count vertex processing, edge inspections, and queue/heap operations. Complexity depends on both representation and algorithm.

$$\sum_{u \in V} \deg(u) = 2|E| \quad \text{for undirected graphs.}$$

Notation / context. Adjacency-list BFS/DFS: $O(V + E)$. Matrix scanning: $O(V^2)$. A sparse simple graph has $E = O(V)$; a dense one can have $E = \Theta(V^2)$.
STOP / caveat. A neighbor loop nested inside a vertex loop can still total $O(V + E)$ rather than $O(V^2)$.

102 ★ Edge relaxation

Meaning. Test whether a path through u improves the best known distance to v. Update the predecessor when it does.

$$\text{if } d[u] + w(u,v) < d[v]: \quad d[v] \leftarrow d[u] + w(u,v), \quad \pi[v] \leftarrow u.$$

Notation / context. Initialize $d[s] = 0$ and other distances to infinity. Shortest-path methods differ in the ordering and repetition of relaxations.
GO / intuition. "Relax" means improve an upper bound on path cost, not finalize it automatically.

103 ★ Dijkstra's algorithm

Meaning. Repeatedly settle the unsettled vertex with smallest tentative source distance, then relax its outgoing edges. Standard correctness requires nonnegative edge weights.

$$u = \arg\min_{v \text{ unsettled}} d[v], \qquad d[u] = \delta(s,u) \text{ when settled.}$$

Notation / context. A min-priority queue selects the next vertex. Store predecessors to reconstruct paths.
STOP / caveat. For one target, stop when it is settled by a valid minimum pop, not merely when first discovered.

104 ★ Dijkstra complexity and stale entries

Meaning. An indexed binary heap supports distance-key updates. A common Python alternative pushes a new entry for each improvement and ignores obsolete entries.

$$T_{\text{indexed}} = O((V+E)\log V), \quad T_{\text{lazy}} = O((V+E)\log(V+E)).$$

Notation / context. Lazy heap test: if popped $dist > d[u]$, skip it. Its heap may hold $O(E)$ entries. Bounds concern reachable processing and conventional graph assumptions.
GO / intuition. Explain the actual implementation rather than quoting a heap bound without its operations.

105 ★ Negative edge weights and Bellman–Ford

Meaning. Negative edges can invalidate Dijkstra's settled-distance guarantee. Bellman–Ford repeatedly relaxes all edges and can detect a reachable negative cycle.

$$T = O(VE), \qquad \text{at most } V - 1 \text{ rounds before cycle detection.}$$

Notation / context. Without a reachable negative cycle, a shortest simple path uses at most $V - 1$ edges. A further improvement signals a reachable negative cycle.
STOP / caveat. Only vertices reachable from such a cycle may have unbounded-below shortest-path cost.

106 ★ A* search

Meaning. Prioritize a search node by known path cost plus estimated remaining cost. A useful heuristic can direct exploration toward the goal.

$$f(n) = g(n) + h(n).$$

Notation / context. $g(n)$: best known cost from source; $h(n)$: heuristic to target; $f(n)$: priority. Update g through relaxation.
STOP / caveat. Optimality depends on assumptions, stopping rule, and duplicate handling. Graph search may need reopening if the heuristic is admissible but inconsistent.

107 ★ Admissible heuristic

Meaning. Never overestimate the true optimal remaining cost. This makes the heuristic optimistic.

$$h(n) \le h^*(n), \qquad h(\text{goal}) = 0 \text{ conventionally.}$$

Notation / context. Straight-line distance is admissible for route length when routes cannot be shorter than geometric straight lines. Units must match edge costs.
STOP / caveat. Admissibility alone does not justify permanently closing every node in all A* graph-search implementations.

108 ★ Consistent heuristic

Meaning. Satisfy a triangle-inequality condition across every edge. Consistency makes A* priorities nondecreasing along a path.

$$h(u) \leq w(u, v) + h(v), \qquad h(\text{goal}) = 0.$$

Notation / context. Under standard conditions, settled nodes need not reopen. Consistency implies admissibility for nodes with finite goal paths.
GO / intuition. An edge cost plus the neighbor's estimated remainder should never be less than the current node's estimated remainder.

109 ★ A* versus Dijkstra and search complexity

Meaning. Setting $h = 0$ makes A* use Dijkstra's priority. Better informed valid heuristics can reduce search, but do not imply a universal logarithmic bound.

$$h(n) = 0 \Rightarrow f(n) = g(n), \qquad \text{implicit tree search can require } O(b^d) \text{ nodes.}$$

Notation / context. b: branching factor; d: depth. Explicit-graph complexity depends on heuristic properties, reopening, and priority-queue implementation.
STOP / caveat. A* can consume substantial memory because it retains a large search frontier.

110 DAG and topological sorting

Meaning. A directed acyclic graph has no directed cycle. A topological order places every edge's source before its destination.

$$u \to v \quad \Rightarrow \quad order(u) < order(v).$$

Notation / context. Kahn's algorithm repeatedly removes indegree-zero vertices; DFS reverse finishing order is another approach. Both take $O(V + E)$ with adjacency lists.
STOP / caveat. If Kahn's algorithm processes fewer than V vertices, a cycle prevents a complete topological ordering.

GREEN STOCK Queues, heaps, priority queues, and core structures

Cards 111–127. Read the local notation before memorizing the equations.

$n,\ k,\ h$	Number of stored items, retained top-k count, heap/tree height.
$i,\ parent(i),\ left(i),\ right(i)$	Array index and implicit binary-tree relationships.
$A[i]$	Heap value/key at index i, using zero-based indexing here.
$priority,\ tie,\ item$	Typical priority-queue tuple fields; tie avoids comparing payloads.
$\deg(u),\ load = n/m$	Vertex degree; hash-table load factor for n entries and m buckets.
h_{tree}	Tree height; do not confuse with A* heuristic $h(n)$.

GO / intuition. FIFO queue → BFS; LIFO stack → DFS; min-priority queue → Dijkstra/A*.

STOP / caveat. A heap is not sorted, and a binary heap is not a binary search tree.

111 ★ Queue

Meaning. FIFO: first in, first out. Enqueue adds to the back; dequeue removes from the front. This preserves discovery order in BFS.

$$enqueue(a), enqueue(b), dequeue() \Rightarrow a.$$

Notation / context. Use a structure with efficient endpoint operations. A linked or circular-buffer queue can support $O(1)$ enqueue/dequeue.
STOP / caveat. Removing the first element of a Python list shifts remaining entries and costs $O(n)$. Use a deque for ordinary BFS.

112 ★ Deque

Meaning. A double-ended queue supports additions and removals at both ends. Python's `collections.deque` is suitable for queues and stacks.

`append`, `appendleft`, `pop`, `popleft` : $O(1)$ endpoint work.

Notation / context. Its middle indexing is $O(n)$, unlike constant-time list indexing. A bounded deque can discard items from the opposite end when full.
GO / intuition. Choose a data structure by required operations, not only by its name.

113 ★ Queue versus stack

Meaning. Queue: FIFO; stack: LIFO. Their removal order determines which discovered state gets explored next.

$$\text{queue} \rightarrow BFS, \qquad \text{stack} \rightarrow DFS.$$

Notation / context. Python list append/pop at the end supports stack use with amortized $O(1)$ operations. Recursion also uses a call stack.
STOP / caveat. A stack does not guarantee minimum-edge paths. A queue alone does not guarantee correct BFS without proper visited handling.

114 ★ Priority queue

Meaning. Remove the item with best priority, rather than earliest arrival. The abstract interface can be implemented by a heap or other structures.

$$extract_min() = \arg\min_{x \in Q} priority(x).$$

Notation / context. Binary-heap priority queue: peek $O(1)$, push/pop $O(\log n)$. Dijkstra prioritizes tentative distance; A* prioritizes $g + h$.
STOP / caveat. Priority ties require an explicit policy when payloads cannot be compared.

115 ★ Binary heap

Meaning. A complete binary tree satisfying heap order. A min-heap makes every parent no greater than its children.

$$A[parent(i)] \leq A[i] \quad (i > 0).$$

Notation / context. The complete shape allows compact array storage and height $\lfloor \log_2 n \rfloor$. The minimum is at the root.
STOP / caveat. Sibling subtrees need not be sorted relative to each other. Arbitrary-value search is generally $O(n)$.

116 ★ Heap array indexing

Meaning. Represent the complete binary tree in a zero-based array. Parent and child positions follow directly from level-order layout.

$$parent(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad left(i) = 2i + 1, \quad right(i) = 2i + 2.$$

Notation / context. Children exist only when their indices are below heap size. The root $i = 0$ has no parent.
GO / intuition. Sift-up follows parent links; sift-down repeatedly compares the current node with its eligible children.

117 ★ Heap operations

Meaning. Insert at the end then sift up; remove the root by moving the last item to the root and sifting down.

$$peek = O(1), \quad push = O(\log n), \quad pop = O(\log n), \quad search = O(n).$$

Notation / context. The logarithmic bound follows from heap height, since an item moves along one root-to-leaf path.
STOP / caveat. Efficient decrease-key requires knowing where the item is; a plain heap array does not provide this lookup automatically.

118 ★ Heapify

Meaning. Build a heap bottom-up, sifting down internal nodes from the last parent toward the root. Most nodes require little movement.

$$\text{work} \le O\left(\sum_{h \ge 0} \frac{n}{2^{h+1}} h\right) = O(n).$$

Notation / context. h: height above leaves. Repeated separate insertions instead have $O(n \log n)$ worst-case total work.
GO / intuition. Many shallow nodes plus very few tall nodes explain why heapify is linear.

119 ★ Top-k with a heap

Meaning. Keep a min-heap of the k largest values encountered. Its root is the weakest retained candidate.

$$x > heap[0] \quad \Rightarrow \quad \text{replace root when heap already has } k \text{ items.}$$

Notation / context. Time $O(n \log k)$ and auxiliary space $O(k)$ for $k \ge 2$; $k = 1$ is a linear scan. Sort retained items if ordered output is required.
STOP / caveat. Keeping the smallest k instead requires the opposite boundary logic, commonly a max-heap.

120 ★ Python heapq

Meaning. Use min-heap operations on a list. `heapify` builds it; `heappush` inserts; `heappop` removes the minimum.

`entry = (priority, counter, payload)`.

Notation / context. A unique counter provides tie-breaking before Python compares payload objects. In lazy Dijkstra, push improved distances and skip stale popped entries.
STOP / caveat. Changing an entry's priority in place can violate heap order. Repair the heap or use an appropriate update strategy.

121 Array and Python list

Meaning. An array stores indexed entries. Python lists are dynamic arrays of object references, allowing growth and heterogeneous referenced objects.

$$\textit{index} = O(1), \quad \textit{append} = O(1) \text{ amortized}, \quad \textit{insert/delete} \text{ near front} = O(n).$$

Notation / context. Resizing occasionally copies references to a larger allocation; geometric growth yields amortized constant-time append.
STOP / caveat. A slice usually creates a new list and takes time and space proportional to slice length.

122 Linked list

Meaning. Nodes hold values and references to neighboring nodes. A singly linked list has next links; a doubly linked list has next and previous links.

$$\textit{search/index} = O(n), \quad \textit{insert} \text{ after a known node} = O(1).$$

Notation / context. Deleting in a singly linked list generally requires its predecessor; a known doubly linked node can usually be removed in $O(1)$.
STOP / caveat. Finding the position may dominate the constant-time pointer update.

123 Hash map and dictionary

Meaning. Map keys to values using a hash function and collision handling. Python dictionaries require hashable keys.

$$lookup/insert/delete = O(1) \text{ expected}, \quad O(n) \text{ worst case.}$$

Notation / context. Load factor is entries divided by bucket capacity; resizing and collisions affect performance. Python dictionaries preserve insertion order.
STOP / caveat. Hash values need not be unique. Equality checks distinguish keys that collide. Mutable contents generally make poor hash keys.

124 Hash set

Meaning. Store unique hashable values for membership tests, duplicate detection, and visited tracking. It uses hashing without an associated application value.

$$contains/add/remove = O(1) \text{ expected.}$$

Notation / context. Expected-time bounds depend on ordinary hashing assumptions; worst cases can be linear. Set membership can replace repeated linear scans through a list.
GO / intuition. Use a set when only presence matters, and a map when you also need counts, positions, or associated data.

125 Binary tree

Meaning. A rooted tree in which every node has at most two children. It does not inherently enforce ordering or balance.

$$\text{full traversal} = O(n), \quad \text{recursive auxiliary space} = O(h).$$

Notation / context. n: nodes; h: height. A chain has $h = \Theta(n)$; a balanced tree has $h = \Theta(\log n)$.
STOP / caveat. Do not claim logarithmic search merely because a structure is binary.

126 Binary search tree

Meaning. A binary tree with an ordering invariant, commonly all left keys smaller and all right keys larger, with an explicit policy for duplicates.

$$search/insert/delete = O(h).$$

Notation / context. Balanced BSTs have $h = O(\log n)$; an unbalanced chain has $h = O(n)$. A heap enforces parent order instead of this subtree-wide ordering.
GO / intuition. Use inorder traversal to obtain sorted keys from a valid BST.

127 Tree traversals and maximum path sum

Meaning. Preorder is root-left-right; inorder left-root-right; postorder left-right-root; level order uses BFS. Postorder supports answers depending on children.

$$gain(v) = v.val + \max(0, gain(v.left), gain(v.right)).$$

Notation / context. For maximum path sum, update a global answer with $v.val + \max(0, g_L) + \max(0, g_R)$, but return only one branch's gain upward.
STOP / caveat. Initialize the global answer to $-\infty$ so all-negative trees are handled correctly.

PINK STOCK Neural networks and CNNs

Cards 128–142. Read the local notation before memorizing the equations.

N, C, H, W	Batch size, channels, height, width. Here W can mean width; context distinguishes it from a weight tensor.
$X \in \mathbb{R}^{N \times C_{in} \times H_{in} \times W_{in}}$	An NCHW image batch; NHWC is another common layout. Always state layout.
$k_h, k_w,\ K_h, K_w$	Common kernel height/width symbols. A bare K may mean kernel size or filter count in a particular source.
$W^{(\ell)}, b^{(\ell)}$	Learned layer weights and bias. A convolution's weight tensor often has shape $C_{out} \times C_{in} \times k_h \times k_w$.
$s_h, s_w,\ p_h, p_w,\ d_h, d_w$	Stride, padding, and dilation by dimension. d here need not mean feature count.
$C_{in}, C_{out},\ F$	Input channels and output channels; F is another common symbol for filter count.
$z^{(\ell)}, a^{(\ell)}, \phi$	Layer preactivation, postactivation, and activation function.
$\delta^{(\ell)}, \odot$	Loss gradient with respect to preactivation and elementwise product.
r_ℓ, j_ℓ	Theoretical receptive-field size and jump between neighboring features in original-input coordinates.
h_t, W_x, W_h	RNN hidden state, input-to-hidden weights, recurrent weights.

STOP / caveat. CNN "convolution" is usually cross-correlation. Kernel dimensions are hyperparameters; kernel entries are learned parameters.

128 Neural network and MLP

Meaning. Compose affine transformations and nonlinear activations. A fully connected multilayer perceptron connects every unit in one layer to every unit in the next.

$$a^{(0)} = x, \quad z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \phi^{(\ell)}(z^{(\ell)}).$$

Notation / context. If widths are $d_{\ell-1}, d_\ell$, then $W^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$. Train all selected weights and biases through the output loss.
STOP / caveat. Stacking only affine layers is still one affine transformation.

129 Activation functions and softmax

Meaning. Nonlinear activations allow complex relationships; output functions match the prediction task.

$$ReLU(x) = \max(0, x), \quad GELU(x) = x\Phi(x), \quad \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

$$p_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad \ell = -\sum_k y_k \log p_k.$$

Notation / context. Φ: standard normal CDF; y: one-hot multiclass target. Softmax plus cross-entropy gives $\partial \ell / \partial z_k = p_k - y_k$.
STOP / caveat. Use independent sigmoids for independent labels, not a sum-to-one softmax.

130 Backpropagation and gradient descent

Meaning. Backpropagation computes derivatives by the chain rule. An optimizer then uses those derivatives to update trainable parameters.

$$\delta^{(\ell)} = (W^{(\ell+1)\top} \delta^{(\ell+1)}) \odot \phi'(z^{(\ell)}), \quad \delta^{(\ell)} = \frac{\partial \ell}{\partial z^{(\ell)}}.$$

$$\frac{\partial \ell}{\partial W^{(\ell)}} = \delta^{(\ell)} a^{(\ell-1)\top}, \quad \frac{\partial \ell}{\partial b^{(\ell)}} = \delta^{(\ell)}, \quad \theta \leftarrow \theta - \eta \nabla_\theta \mathcal{J}.$$

Notation / context. Equations show one example; minibatch gradients aggregate across examples.

131 ★ Convolutional neural network

Meaning. Use learned local filters shared across spatial positions. Early features can detect edges/textures; later layers combine them into broader patterns.

$$X \xrightarrow{\text{conv + activation}} A \xrightarrow{\text{more layers}} \hat{y}.$$

Notation / context. A classification CNN commonly ends in pooled features, class logits, softmax, and cross-entropy. Convolution weights receive gradients through every spatial use.
GO / intuition. Local connectivity and parameter sharing encode the assumption that useful visual patterns can recur at different locations.

132 ★ Convolution, filter, and kernel

Meaning. A filter combines a local input region across channels. The following stride-one expression uses the cross-correlation convention.

$$Z_{u,v,o} = b_o + \sum_{c=1}^{C_{in}} \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} W_{o,c,a,b} X_{u+a,v+b,c}.$$

Notation / context. u, v: spatial position; o: output channel/filter; c: input channel; a, b: kernel offsets. Input channel-last notation is used here for readability; batch omitted.
STOP / caveat. The sum-index b differs from bias b_o by context. No kernel flipping occurs here.

133 ★ Feature maps and channels

Meaning. Each learned filter produces a spatial response map. Stacking maps produces the output channels. Input channels describe measurements or preceding learned features.

$$N \times C_{in} \times H \times W \longrightarrow N \times C_{out} \times H_{out} \times W_{out}.$$

Notation / context. An RGB image has $C_{in} = 3$. A standard layer with 16 filters gives $C_{out} = 16$, not 48 output channels.
GO / intuition. Each standard filter spans all input channels; filter count controls output channels. Grouped/depthwise convolutions modify this rule.

134 ★ Parameter sharing and local connectivity

Meaning. Local connectivity limits each output to a nearby region. Sharing uses the same filter weights at every eligible location.

$$\frac{\partial \mathcal{L}}{\partial W_{o,c,a,b}} = \sum_{i,u,v} \frac{\partial \mathcal{L}}{\partial Z_{i,u,v,o}} X_{i,u+a,v+b,c}.$$

Notation / context. Shown for stride one without padding offsets; i indexes batch examples. Gradients sum contributions from all uses of a weight.
GO / intuition. One pattern detector is learned from many positions instead of assigning independent parameters to every location.

135 ★ Stride

Meaning. Stride specifies the step between consecutive filter placements. Larger stride downsamples the spatial output.

$$Z_{u,v,o} = b_o + \sum_{c,a,b} W_{o,c,a,b} X_{s_h u+a,\ s_w v+b,\ c}.$$

Notation / context. s_h, s_w: vertical/horizontal strides; padding and dilation omitted here. Stride two roughly halves each spatial dimension in common padded configurations.
STOP / caveat. Downsampling discards spatial detail and can introduce aliasing; computational savings are not free of modeling consequences.

136 ★ Padding, dilation, and output size

Meaning. Padding extends boundaries; dilation spaces out kernel sample positions. For one dimension with symmetric padding:

$$n_{out} = \left\lfloor \frac{n + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

Notation / context. n: input length; p: padding per side; d: dilation; k: kernel length; s: stride. Effective kernel size is $d(k-1)+1$.
GO / intuition. Example: $n = 32, k = 3, p = 1, d = 1$ gives 32 outputs for $s = 1$ and 16 for $s = 2$.

137 ★ Receptive field

Meaning. The input region that can influence an activation. Stacking layers enlarges the theoretical receptive field.

$$r_\ell = r_{\ell-1} + (k_\ell - 1) d_\ell j_{\ell-1}, \quad j_\ell = j_{\ell-1} s_\ell, \quad r_0 = j_0 = 1.$$

Notation / context. r: receptive-field size; j: spacing in original-input coordinates; k, d, s: kernel, dilation, stride. Formula applies per dimension.
GO / intuition. Three stride-one 3-wide convolutions have receptive-field sizes 3, 5, and 7. Effective influence can be smaller than theoretical support.

138 ★ Pooling

Meaning. Summarize local regions without standard learned filter weights. Max pooling selects the largest activation; average pooling takes the mean.

$$Y_{u,v,c} = \max_{(a,b)\in\mathcal{R}_{u,v}} X_{a,b,c}, \quad Y^{avg}_{u,v,c} = \frac{1}{|\mathcal{R}_{u,v}|} \sum_{(a,b)\in\mathcal{R}_{u,v}} X_{a,b,c}.$$

Notation / context. $\mathcal{R}_{u,v}$: input region associated with an output. Max-pool gradients route through selected maxima, subject to tie conventions.
STOP / caveat. Pooling reduces detail and does not guarantee complete translation invariance.

139 ★ Translation equivariance versus invariance

Meaning. Equivariance means a transformed input produces a correspondingly transformed representation. Invariance means the representation stays unchanged.

$$f(Tx) = Tf(x) \quad \text{equivariance;} \qquad f(Tx) = f(x) \quad \text{invariance.}$$

Notation / context. T: a translation operator. Convolution is equivariant under suitable boundary and sampling conditions; global aggregation can encourage invariant outputs.
STOP / caveat. Padding boundaries and strided sampling complicate exact equivariance for arbitrary shifts.

140 ★ CNN parameter count and compute

Meaning. Standard convolution parameters depend on filter shape and channel counts, while computation also depends on output spatial size.

$$P = k_h k_w C_{in} C_{out} + C_{out}, \quad MACs \approx H_{out} W_{out} C_{out} k_h k_w C_{in}.$$

Notation / context. Bias included; multiply-accumulate count is per example. A 3×3 layer with 3 input and 16 output channels has $3 \cdot 3 \cdot 3 \cdot 16 + 16 = 448$ parameters.
STOP / caveat. Larger images can increase compute without changing parameter count.

141 Recurrent neural network

Meaning. Process a sequence with a shared state-update rule. The hidden state carries information from preceding steps.

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h), \quad z_t = W_y h_t + b_y.$$

Notation / context. t: sequence position; h_t: hidden state. Fit with a task loss and backpropagation through time; repeated Jacobian products can cause vanishing/exploding gradients.
GO / intuition. Unlike an MLP on a fixed vector, the recurrent rule explicitly reuses state across positions.

142 Transfer learning and fine-tuning

Meaning. Reuse pretrained representations. Freezing the backbone trains only new components; fine-tuning updates selected pretrained parameters as well.

$$\hat{y} = h_\psi(g_\phi(x)), \quad \text{frozen backbone: update } \psi \text{ only}; \quad \text{fine-tune: update selected } \phi, \psi.$$

Notation / context. g_ϕ: pretrained encoder; h_ψ: task head. Use task loss, representative validation, and often a smaller learning rate for pretrained parameters.
STOP / caveat. Changing an item encoder can require recomputing stored item embeddings and rebuilding retrieval indexes.

ORANGE STOCK Applied ML, language, recommendation, generation, and RL

Cards 143–157. Read the local notation before memorizing the equations.

$u, v,\ e_u, e_v$	User/item or query/item identities and embeddings. Some sources use u, v directly for the vectors.
$s(u, v),\ \tau$	Compatibility score and contrastive temperature; this τ is not a decision threshold.
$q,\ k,\ rel_i,\ r_q$	Query, ranking cutoff, relevance at position i, first relevant rank for query q.
Q, K, V	Attention query, key, and value matrices. Here K is not an SVM kernel or a CNN kernel size.
$d_k,\ T,\ \mathcal{V}$	Attention key dimension, sequence length, vocabulary.
$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$	MDP: states, actions, transitions, rewards, discount factor.
$\pi_\theta(a \mid s),\ G_t,\ V^\pi(s),\ Q^\pi(s, a)$	Policy, return, state value, action value. Q^π differs from attention's Q.
$\alpha, \eta, \gamma, \varepsilon$	Often RL learning rate(s), discount factor, exploration probability; always define locally.

GO / intuition. For your fashion recommender: specify the user/item inputs, compatibility score, observed feedback, training loss, and retrieval/ranking metrics.

143 Applied machine learning

Meaning. Turn a real problem into a measurable prediction-and-decision system. Data validity, operational constraints, and outcome costs are part of the task.

$$a^*(x) = \arg\min_a \mathbb{E}[C(a, Y) \mid X = x].$$

Notation / context. a: action; C: action/outcome cost. A model's probabilities support a decision rule, but offline loss is not automatically the business objective.
GO / intuition. Define target, prediction time, baseline, split, metric, deployment constraints, and monitoring before proposing a complex architecture.

144 Computer vision tasks

Meaning. Classification labels an image; detection predicts boxes and classes; segmentation labels pixels or regions. Video also requires temporal reasoning.

$$\text{box} = (x_{min}, y_{min}, x_{max}, y_{max}), \quad IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Notation / context. A, B: predicted/true regions. Detection losses often combine classification and localization terms; segmentation often uses pixelwise cross-entropy or overlap-based losses.
STOP / caveat. An image-level label does not supply the localization annotations required by supervised detection.

145 Tokenization and embeddings

Meaning. Tokenization maps text into vocabulary IDs. An embedding table maps each ID to a learned vector.

$$E \in \mathbb{R}^{|\mathcal{V}| \times d}, \quad e_t = E[token_t], \quad \cos(u, v) = \frac{u^\top v}{\|u\|\|v\|}.$$

Notation / context. $|\mathcal{V}|$: vocabulary size; d: embedding dimension. Embedding parameters learn through the task loss; contextual models further transform these vectors.
STOP / caveat. Vector similarity depends on training and normalization; it is not proof of semantic equivalence.

146 Transformer and self-attention

Meaning. Each position combines information from other permitted positions using learned query, key, and value projections.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Notation / context. X: input representations; d_k: key dimension; M: mask, with $-\infty$ on forbidden interactions. Softmax acts across key positions. Projections learn by backpropagation.
STOP / caveat. Order requires positional information; causal masking prevents access to future tokens.

147 Generative versus discriminative models

Meaning. Generative models describe distributions used to generate observations or outputs. Discriminative models directly predict labels or decisions from inputs.

$$\text{generative: } p_\theta(x) \text{ or } p_\theta(x \mid c); \qquad \text{discriminative: } p_\theta(y \mid x) \text{ or } f_\theta(x).$$

Notation / context. Likelihood training minimizes $-\sum_i \log p_\theta(x_i)$ when tractable. Conditional generation predicts structured outputs given context c.
STOP / caveat. The terminology depends on the task; conditional probability notation alone does not settle every architectural classification.

148 Autoregressive language model

Meaning. Generate a sequence one token at a time, conditioning each next-token distribution on the preceding context.

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}), \quad \mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}).$$

Notation / context. T: sequence length; token probabilities usually come from softmax vocabulary logits. Train with observed prefixes; generation uses previously generated tokens.
GO / intuition. Learning increases probability assigned to the observed next tokens through gradient-based updates.

149 Retrieval-augmented generation

Meaning. Retrieve external material and provide it as context to a generator. The retriever may be separately trained, frozen, or jointly optimized in some systems.

$$D_q = \mathit{Retrieve}(q), \qquad \hat{y} \sim p_\theta(y \mid q, D_q).$$

Notation / context. q: query; D_q: retrieved passages; y: generated answer. Evaluate retrieval quality separately from grounded answer quality.
STOP / caveat. RAG does not guarantee truth: retrieval can omit evidence and generation can misinterpret or contradict it.

150 Retrieval versus ranking

Meaning. Retrieval selects a candidate set efficiently from a large collection. Ranking orders that set using detailed relevance or utility scores.

$$C(q) = \mathit{Retrieve}(q), \qquad \mathit{ranked}(q) = \operatorname{sort}_{v \in C(q)} s(q, v).$$

Notation / context. $C(q)$: candidates. Retrieval recall limits what later ranking can recover. A heavier ranker can be affordable after the candidate set shrinks.
GO / intuition. Explain why candidate generation and final ordering can require different models and metrics.

151 Two-tower recommender

Meaning. Encode users/queries and items separately, then compare embeddings. Independent item encoding enables precomputation and efficient vector retrieval.

$$e_u = f_\theta(x_u), \quad e_v = g_\phi(x_v), \quad s = e_u^\top e_v, \quad p = \sigma(s + b).$$
$$\ell_{BCE} = -[y \log p + (1 - y) \log(1 - p)].$$

Notation / context. Update trainable parts of both towers through the loss. For your fashion system, item inputs can be image/text features and user inputs can be preferences/history.

152 Implicit feedback and contrastive learning

Meaning. Clicks, watch time, and purchases provide indirect preferences. Contrastive training makes observed positive pairs score above sampled alternatives.

$$\ell = -\log \frac{\exp(s(u, v^+)/\tau)}{\exp(s(u, v^+)/\tau) + \sum_j \exp(s(u, v_j^-)/\tau)}.$$

Notation / context. v^+: positive; v_j^-: sampled negatives; $\tau > 0$: temperature. Cold start can use item content and initial user preferences.
STOP / caveat. Missing feedback is not proven dislike. Hard negatives may be useful but can also be false negatives.

153 Precision@k, Recall@k, Hit@k

Meaning. Measure relevance among top-ranked results. Let rel_i be binary relevance and R the relevant-item set.

$$P@k = \frac{\sum_{i=1}^{k} rel_i}{k}, \quad R@k = \frac{\sum_{i=1}^{k} rel_i}{|R|}, \quad Hit@k = \mathbf{1}\left[\sum_{i=1}^{k} rel_i > 0\right].$$

Notation / context. Average per-query/user results as specified. Hit@k commonly means any relevant result occurs within the cutoff.
STOP / caveat. Define behavior for zero relevant items and lists shorter than k; conventions vary.

154 MRR and NDCG

Meaning. MRR rewards an early first relevant result. NDCG rewards placing multiple, potentially graded, relevant items near the top.

$$MRR = \frac{1}{Q}\sum_{q=1}^{Q}\frac{1}{r_q}, \quad DCG@k = \sum_{i=1}^{k}\frac{2^{rel_i}-1}{\log_2(i+1)}, \quad NDCG@k = \frac{DCG@k}{IDCG@k}.$$

Notation / context. r_q: first relevant rank, zero reciprocal contribution if absent; $IDCG$: ideal DCG for that query. State a convention when $IDCG = 0$.
GO / intuition. MRR focuses on the first hit; NDCG can distinguish graded quality across the list.

155 Reinforcement learning and MDP

Meaning. Learn behavior through interaction to maximize expected cumulative reward. An MDP specifies states, actions, transition dynamics, rewards, and discounting.

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \quad J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G_0].$$

Notation / context. $P(s' \mid s, a)$: transition law; τ: trajectory, not contrastive temperature here. The Markov state contains the information needed to predict the next transition.
GO / intuition. RL optimizes actions and their future consequences, rather than merely predicting a supplied label.

156 Policy, reward, return, and discount

Meaning. A policy selects actions; reward is immediate feedback; return accumulates future rewards.

$$a_t \sim \pi_\theta(a \mid s_t), \qquad G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

Notation / context. $\gamma \in [0, 1)$ for standard discounted infinite-horizon returns. Finite episodes can use $\gamma = 1$ when returns remain well-defined.
GO / intuition. One policy-gradient form is $\nabla_\theta J = \mathbb{E}[\sum_t \gamma^t G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)]$ under standard conditions; subtracting a suitable baseline reduces variance.

157 Value, Q-learning, and exploration

Meaning. $V^\pi(s)$ is expected return from a state; $Q^\pi(s, a)$ also fixes the initial action. Learning balances exploration with exploiting current estimates.

$$Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big].$$

Notation / context. α: update rate; bracket: temporal-difference error. For a terminal next state, the bootstrap term is zero. ε-greedy chooses a random action with probability ε.
GO / intuition. Bellman optimality: $Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a') \mid s, a]$.

Card index

Card	Stock	Front-of-card term
1	WHITE	Supervised learning
2	WHITE	Unsupervised learning
3	WHITE	Classification
4	WHITE	Regression
5	WHITE	Features and labels
6	WHITE	Parameters versus hyperparameters
7	WHITE	Train, validation, and test sets
8	WHITE	Cross-validation
9	WHITE	Generalization
10	WHITE	Bias
11	WHITE	Variance
12	WHITE	Bias–variance decomposition
13	WHITE	Overfitting
14	WHITE	Underfitting
15	WHITE	L1 and L2 regularization
16	WHITE	Early stopping
17	WHITE	Data leakage
18	WHITE	Class imbalance
19	WHITE	Confusion matrix
20	WHITE	Accuracy
21	WHITE	Precision
22	WHITE	Recall
23	WHITE	F1 score
24	WHITE	Decision threshold
25	WHITE	ROC curve and ROC-AUC
26	WHITE	Precision–recall curve
27	WHITE	Calibration
28	WHITE	Linear regression
29	WHITE	Logistic regression
30	WHITE	Sigmoid
31	WHITE	Cross-entropy, likelihood, and KL
32	WHITE	Decision tree
33	WHITE	Gini impurity and entropy
34	WHITE	Bagging and random forest
35	WHITE	Gradient boosting
36	WHITE	★ Support vector machine
37	WHITE	★ Margin and support vectors
38	WHITE	★ Kernel trick
39	WHITE	★ SVM training: C, gamma, and alpha
40	WHITE	Distribution shift and concept drift
41	WHITE	Object-oriented design
42	WHITE	Object-oriented programming
43	WHITE	Class versus object
44	WHITE	Attribute versus method
45	WHITE	Object creation and initialization
46	WHITE	Encapsulation
47	WHITE	Abstraction
48	WHITE	Inheritance
49	WHITE	Composition
50	WHITE	Polymorphism
51	WHITE	Interface and protocol
52	WHITE	Single-responsibility principle
53	WHITE	Cohesion versus coupling
54	WHITE	Dependency injection
55	WHITE	API
56	WHITE	HTTP API and endpoint

Card	Stock	Front-of-card term
57	WHITE	HTTP methods
58	WHITE	Idempotency
59	WHITE	Exception handling
60	WHITE	Mutability and aliasing
61	WHITE	Unit testing
62	WHITE	Integration testing
63	WHITE	End-to-end testing
64	WHITE	Test case
65	WHITE	Edge case
66	WHITE	Corner case
67	WHITE	Boundary-value analysis
68	WHITE	Equivalence partitioning
69	WHITE	Assertion
70	WHITE	Test oracle
71	WHITE	Property-based testing
72	WHITE	Mock, stub, and fake
73	WHITE	Regression testing
74	WHITE	Invariant
75	WHITE	Preconditions and postconditions
76	YELLOW	Big-O, Big-Theta, and Big-Omega
77	YELLOW	Time versus space complexity
78	YELLOW	Worst, average, and amortized analysis
79	YELLOW	★ Comparison sorting lower bound
80	YELLOW	★ Stable and in-place sorting
81	YELLOW	★ Merge sort
82	YELLOW	★ Quicksort
83	YELLOW	★ Heapsort
84	YELLOW	★ Insertion sort
85	YELLOW	★ Counting sort
86	YELLOW	Binary search
87	YELLOW	★ Divide and conquer
88	YELLOW	★ Recurrences and Master theorem
89	YELLOW	Recursion
90	YELLOW	Backtracking
91	YELLOW	Dynamic programming
92	YELLOW	Memoization versus tabulation
93	YELLOW	Greedy algorithms
94	YELLOW	0/1 knapsack
95	YELLOW	P, NP, NP-hard, NP-complete
96	BLUE	Graph terminology
97	BLUE	★ Adjacency list
98	BLUE	★ Adjacency matrix
99	BLUE	★ Breadth-first search
100	BLUE	★ Depth-first search
101	BLUE	★ Runtime analysis of graphs
102	BLUE	★ Edge relaxation
103	BLUE	★ Dijkstra's algorithm
104	BLUE	★ Dijkstra complexity and stale entries
105	BLUE	★ Negative edge weights and Bellman–Ford
106	BLUE	★ A* search
107	BLUE	★ Admissible heuristic
108	BLUE	★ Consistent heuristic
109	BLUE	★ A* versus Dijkstra and search complexity
110	BLUE	DAG and topological sorting
111	GREEN	★ Queue
112	GREEN	★ Deque
113	GREEN	★ Queue versus stack
114	GREEN	★ Priority queue
115	GREEN	★ Binary heap

Card	Stock	Front-of-card term
116	GREEN	★ Heap array indexing
117	GREEN	★ Heap operations
118	GREEN	★ Heapify
119	GREEN	★ Top-k with a heap
120	GREEN	★ Python heapq
121	GREEN	Array and Python list
122	GREEN	Linked list
123	GREEN	Hash map and dictionary
124	GREEN	Hash set
125	GREEN	Binary tree
126	GREEN	Binary search tree
127	GREEN	Tree traversals and maximum path sum
128	PINK	Neural network and MLP
129	PINK	Activation functions and softmax
130	PINK	Backpropagation and gradient descent
131	PINK	★ Convolutional neural network
132	PINK	★ Convolution, filter, and kernel
133	PINK	★ Feature maps and channels
134	PINK	★ Parameter sharing and local connectivity
135	PINK	★ Stride
136	PINK	★ Padding, dilation, and output size
137	PINK	★ Receptive field
138	PINK	★ Pooling
139	PINK	★ Translation equivariance versus invariance
140	PINK	★ CNN parameter count and compute
141	PINK	Recurrent neural network
142	PINK	Transfer learning and fine-tuning
143	ORANGE	Applied machine learning
144	ORANGE	Computer vision tasks
145	ORANGE	Tokenization and embeddings
146	ORANGE	Transformer and self-attention
147	ORANGE	Generative versus discriminative models
148	ORANGE	Autoregressive language model
149	ORANGE	Retrieval-augmented generation
150	ORANGE	Retrieval versus ranking
151	ORANGE	Two-tower recommender
152	ORANGE	Implicit feedback and contrastive learning
153	ORANGE	Precision@k, Recall@k, Hit@k
154	ORANGE	MRR and NDCG
155	ORANGE	Reinforcement learning and MDP
156	ORANGE	Policy, reward, return, and discount
157	ORANGE	Value, Q-learning, and exploration